

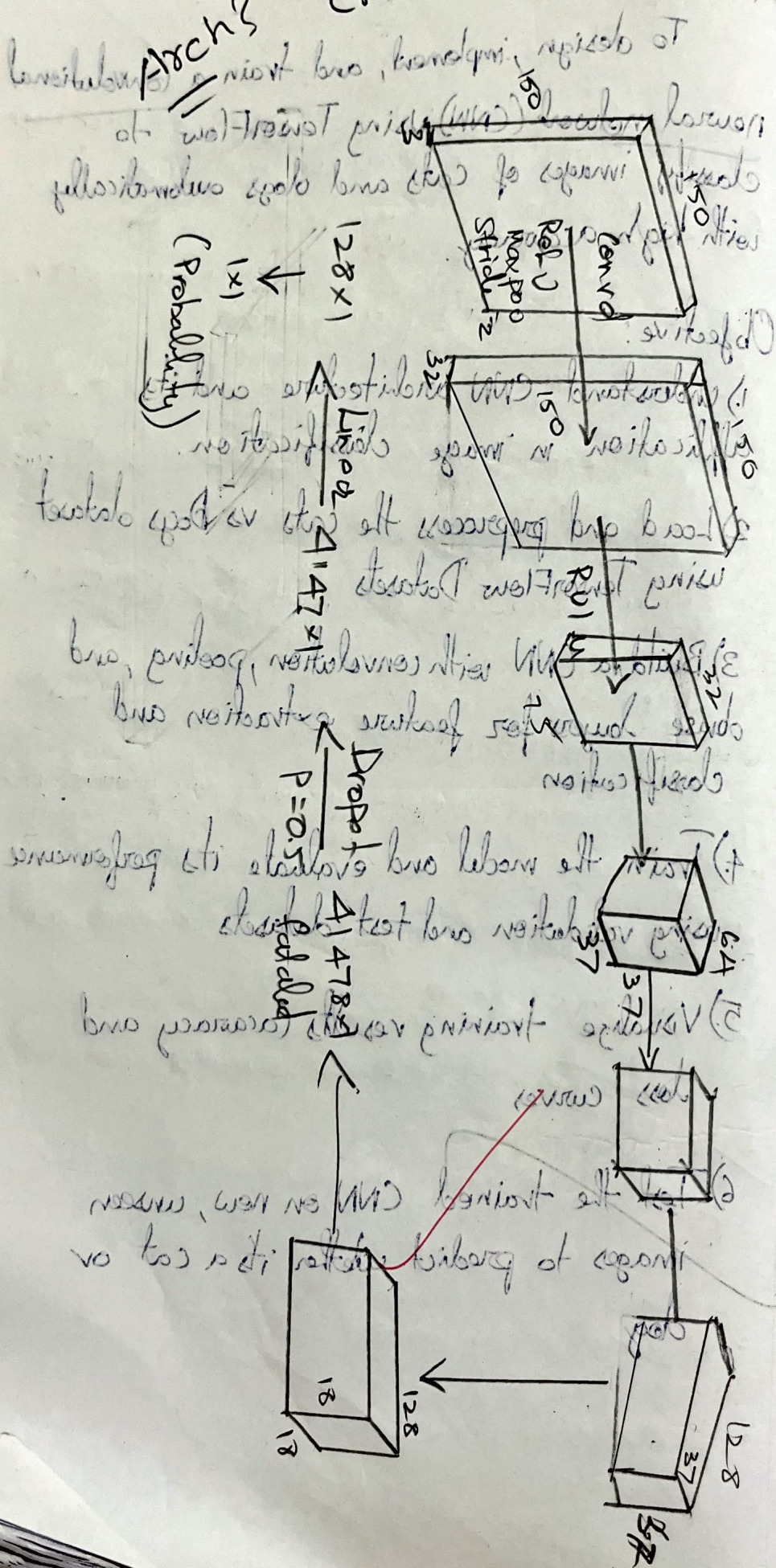
Aim:-

To design, implement, and train a Convolutional neural network (CNN) using TensorFlow to classify images of cats and dogs automatically with high accuracy.

Objective:

- 1) Understand CNN architecture and its application in image classification.
- 2) Load and preprocess the cats vs Dogs dataset using TensorFlow Datasets
- 3) Build a CNN with convolution, pooling, and dense layers for feature extraction and classification
- 4) Train the model and evaluate its performance using validation and test datasets
- 5) Visualize training results (accuracy and loss curves)
- 6) Test the trained CNN on new, unseen images to predict whether it's a cat or dog

Arch 3



pseudo Code:

1. Import libraries

- tensorflow, tensorflow-datasets, matplotlib

2. LOAD DATASET

- use tfds.load('cats-vs-dogs')
- split data into train, validation, and test sets

3. DEFINE preprocess_image FUNCTION

- Resize image to fixed size (e.g. 160x160)
- Normalize pixel values to range [0,1]

4. APPLY preprocessing to dataset

- Map preprocess_image to train, validation, test sets
- Shuffle and batch the training dataset

5. BUILD CNN MODEL

- Sequential model:

Conv2D(32 filters, 3x3, relu) + MaxPooling(2x2)

Conv2D(64 filters, 3x3, relu) + MaxPooling(2x2)

Conv2D(128 filters, 3x3, relu) + Maxpooling(2x2)

Flatten

Dense(128, relu) + Dropout(0.5)

Dense(1, sigmoid)

6. COMPILE MODEL

- Optimizer = Adam
- Loss = Binary Crossentropy
- Metrics = Accuracy

2. Build CNN Model

Validation Acc
Train Acc

difficult, chaotic - w/loss, w/loss

LOAD DATASET

split data into train, validation, and test

3. DEFINE LOSS FUNCTION

- Raise image of fixed size (e.g. 100x100)

- Normalize fixed values to range [0,1]

4. APPLY PREPROCESSING TO DATASET

- Map features - image to train, validation, test sets

- Shuffle and batch the training dataset

2. BUILD CNN MODEL

- sequential model:

0 2 4 6 8

(conv2d(32, filter(3x3, relu) + maxpooling(2x2))

(conv2d(64, filter(3x3, relu) + maxpooling(2x2))

(conv2d(128, filter(3x3, relu) + maxpooling(2x2))

Flatten

Dense(128, relu) + Dropout(0.2)

Dense(1, sigmoid)

2. COMPILE MODEL

- Optimizer = Adam

- Loss = Binary Crossentropy

- Metrics = Accuracy

7. TRAIN MODEL

- Fit model on training dataset
- Validate on validation dataset
- set epoch (eg., 10)

8. Evaluate MODEL

- Evaluate on test dataset
- Print test accuracy

9. VISUALIZE RESULTS

- Plot training and validation accuracy
- Plot training and validation loss

10. PREDICT ON SINGLE IMAGE

- Load image
- Preprocess image
- model.predict(image)
- If prediction > 0.5 $\rightarrow$ Dog else Cat

END

Confusion matrix

Observation:-

Accuracy : 82.59 %

~~Result :~~

The code successfully executed.